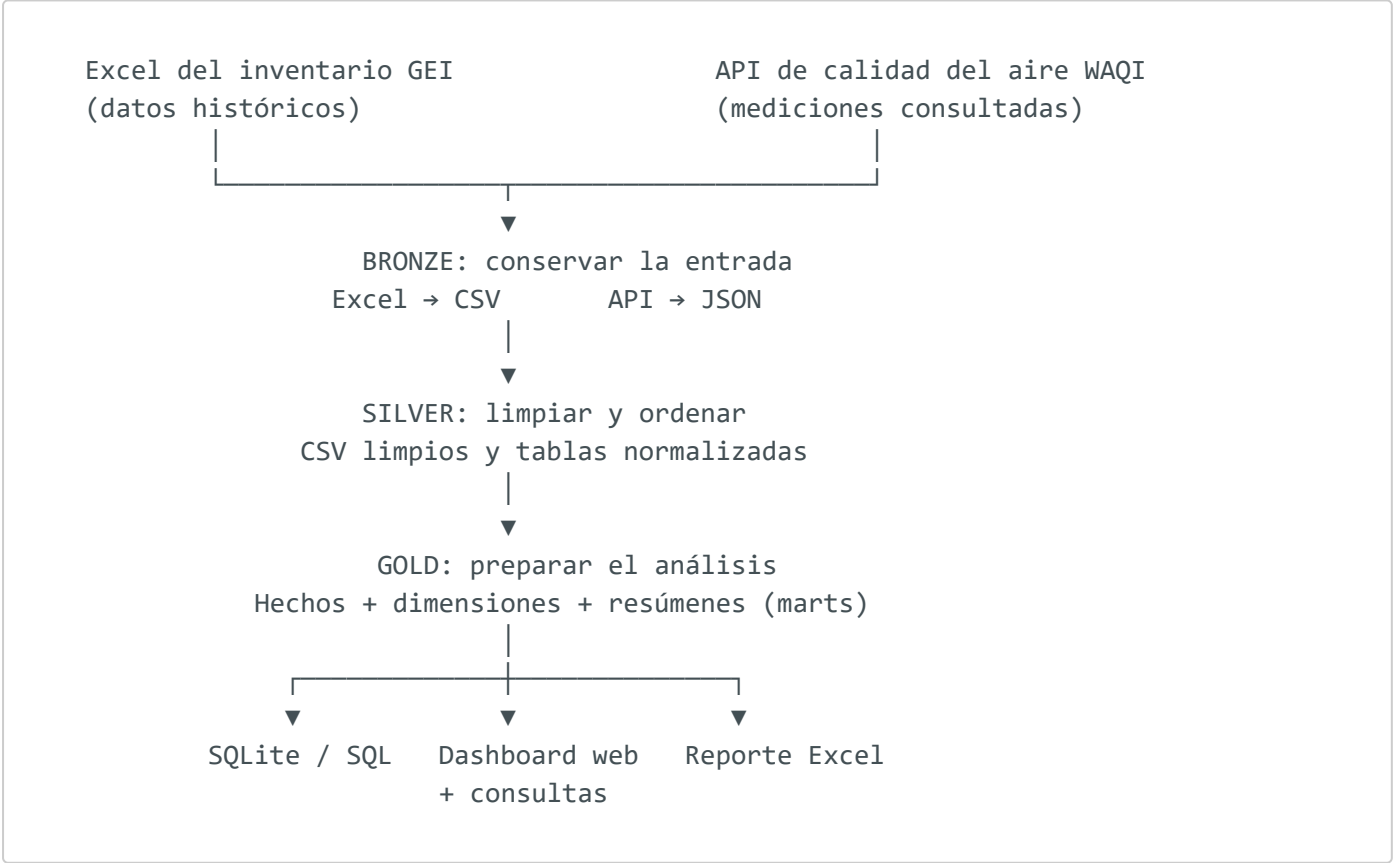
Guía sencilla del proyecto Green Data ETL

Esta guía explica qué hace el proyecto, cómo viajan los datos y cómo presentarlo a una persona que no conoce el código.

1. La idea en una frase

El proyecto toma el inventario histórico de gases de efecto invernadero de Colombia y mediciones de calidad del aire, los limpia y organiza, y los presenta en un dashboard, una base de datos SQLite y un reporte Excel.

2. Vista rápida: del dato al análisis



Importante: la API de WAQI se consulta cuando se ejecuta la extracción Bronze. El proyecto conserva una respuesta de ese momento; no mantiene un flujo continuo de datos en vivo.

3. ¿Qué significa ETL?

- **E — Extraer:** leer el Excel y consultar la API WAQI.
- **T — Transformar:** corregir, limpiar, normalizar y resumir los datos.
- **L — Cargar:** guardar los resultados listos para análisis en SQLite y Excel.

La estructura **Bronze** → **Silver** → **Gold** organiza esas transformaciones por etapas. Cada capa tiene un propósito distinto.

4. Las tres capas

Capa	En palabras sencillas	Entrada principal	Resultado
Bronze	Copia de los datos recibidos, con poca modificación	Archivo Excel y API WAQI	`data/bronze/excel/raw_gei_inventory.csv` y `data/bronze/api/raw_waqi_api_extract.json`
Silver	Datos corregidos y ordenados para que tengan una estructura coherente	CSV y JSON de Bronze	Cinco archivos CSV limpios en `data/silver/`
Gold	Tablas y resúmenes diseñados para responder preguntas	CSV de Silver	Tablas analíticas en `data/gold/`, una base SQLite y un informe Excel

Bronze: conservar las fuentes

1. El Excel del inventario se encuentra en la raíz del proyecto; su ruta está definida en `config/settings.py`.
2. `src/extract/excel_extractor.py` lee el Excel, agrega datos técnicos de auditoría y guarda el CSV Bronze.
3. `src/extract/api_extractor.py` consulta WAQI, busca estaciones de algunas ciudades colombianas y guarda las respuestas originales en JSON.

El registro de Excel incluye información como el momento de extracción y una huella SHA-256 del archivo para identificar la fuente.

Silver: reparar y conformar

`src/transform/bronze_to_silver.py` procesa las dos fuentes:

- **Inventario GEI:** estandariza nombres de columnas, repara algunos textos mal codificados, identifica códigos y niveles IPCC y convierte emisiones por gas a registros separados.
- **Balance de emisiones:** comprueba la relación entre emisiones brutas, absorciones y emisiones netas. En este inventario, las absorciones suelen ser valores negativos; por eso la fórmula implementada es `netas = brutas + absorciones`.
- **Calidad del aire:** extrae de la respuesta anidada de WAQI los datos de las estaciones y sus mediciones, como AQI, PM2.5, PM10 y variables meteorológicas.

Los cinco archivos Silver son:

Archivo	Contenido
`silver_inventario_gei_ancho.csv`	Inventario limpio en formato ancho, similar a la tabla de entrada
`silver_inventario_gei_normalizado.csv`	Hechos de emisiones con una fila por año, sector y gas
`silver_jerarquia_sectores.csv`	Catálogo de sectores y jerarquía IPCC
`silver_estaciones_calidad_aire.csv`	Catálogo de estaciones, ciudades y coordenadas

Archivo	Contenido
`silver_mediciones_calidad_aire.csv`	Mediciones y contaminantes registrados por estación

Gold: facilitar las preguntas analíticas

`src/transform/silver_to_gold.py` crea tablas organizadas para combinar y resumir datos. En el modelo dimensional:

- Una **tabla de hechos** guarda observaciones o valores medibles, por ejemplo emisiones o mediciones de aire.
- Una **dimensión** describe esas observaciones: el año, el sector, el gas o la estación.
- Un **data mart** contiene un resumen pensado para una pregunta de negocio, como el total por año y macro-sector o los indicadores ejecutivos.

Entre las salidas Gold están:

- `fact_emisiones_gei_historico.csv`
- `fact_calidad_aire_tiempo_real.csv`
- `dim_tiempo.csv`
- `dim_sector_gei.csv`
- `dim_tipo_gas_contaminante.csv`
- `dim_estacion_monitoreo.csv`
- `mart_consolidado_ambiental.csv`
- `mart_kpis_resumen_ejecutivo.csv`

Los mismos modelos se cargan a `data/gold/green_data_warehouse.db` y se usan para generar `data/gold/reporte_ejecutivo_green_data.xlsx`.

5. ¿Por qué se guardan datos en SQL?

La respuesta de WAQI llega en JSON, una estructura útil para transportar y conservar la respuesta original. Pero para hacer preguntas analíticas conviene tener los valores organizados en columnas y tablas.

El proyecto prepara esos datos en Silver y Gold y usa **SQLite**, una base de datos guardada en un archivo local. Así se pueden hacer consultas como:

```sql
SELECT anio, sector_macro_nombre, emisiones_brutas_kt
FROM mart_consolidado_ambiental
ORDER BY anio DESC
LIMIT 10;
```

SQL no reemplaza el JSON de Bronze: ambos sirven para cosas distintas. Bronze preserva la respuesta de origen; Gold facilita consultar los datos organizados. La interfaz, además, puede pedir datos ya resumidos a la base sin volver a procesar toda la extracción.

6. ¿Qué hace cada parte del programa?

Ruta	Responsabilidad
`config/settings.py`	Rutas, variables de entorno, parámetros WAQI y catálogos usados por el pipeline
`src/extract/`	Obtiene los datos de Excel y WAQI para Bronze
`src/transform/bronze_to_silver.py`	Limpia y normaliza Bronze a Silver
`src/transform/silver_to_gold.py`	Construye dimensiones, hechos y data marts
`src/load/database_loader.py`	Escribe Gold en SQLite y genera el Excel
`src/utils/quality_checker.py`	Comprueba reglas de calidad e integridad
`src/api/server.py`	Backend FastAPI: coordina el pipeline y ofrece datos al frontend
`frontend/`	Interfaz web React/Vite para ejecutar etapas, ver resultados, consultar SQL y usar el chat
`test/test_etl.py`	Pruebas automatizadas del pipeline
`notebooks/`	Notebook de análisis exploratorio

La interfaz y el backend

La aplicación web no transforma los datos por sí sola. Envía solicitudes al backend:

1. El frontend React muestra los botones y paneles.
2. FastAPI, en `src/api/server.py`, recibe solicitudes como ejecutar Bronze, Silver o Gold.
3. El backend llama a los extractores, transformadores y al cargador correspondientes.
4. Para el dashboard, el backend consulta Gold y devuelve resultados en JSON.

El chat es una función aparte: el backend prepara un resumen de los datos Gold y lo envía a Ollama. Ollama genera una respuesta en lenguaje natural; no ejecuta ni sustituye el pipeline ETL.

7. Cómo iniciar el proyecto en Windows

Abre **dos terminales** en la carpeta raíz del proyecto. Si el entorno virtual todavía no está activo, actívalo en cada terminal:

```
.\.venv\Scripts\Activate.ps1
```

Si todavía no instalaste los paquetes de Python:

```
python -m pip install -r requirements.txt
```

Terminal 1: iniciar el backend

```
python -m uvicorn src.api.server:app --port 8000 --reload
```

Terminal 2: iniciar la interfaz web

```
cd frontend
npm install
npm run dev
```

Abre en el navegador la dirección que Vite muestre en la terminal; normalmente es `http://localhost:3000`.

Orden para generar los datos

En el panel de la aplicación, ejecuta:

```
1. Bronze  →  2. Silver  →  3. Gold
```

- **Bronze** necesita el archivo Excel de origen y acceso a Internet para consultar WAQI.
- **Silver** necesita los archivos Bronze.
- **Gold** necesita los archivos Silver.
- Ollama solo es necesario para el chat, no para ejecutar las tres capas ETL.

Para ejecutar las pruebas documentadas:

```
python -m unittest test/test_etl.py
```

La suite puede consultar WAQI y leer el Excel, así que depende de la disponibilidad de esos recursos.

8. ¿Cómo se verifica que funcionó?

- La interfaz muestra el estado de cada capa y el resultado de la última ejecución.
- Los registros de ejecución se guardan en `data/pipeline_execution.log`; el servidor también escribe `data/api_server.log`.
- En `data/bronze/`, `data/silver/` y `data/gold/` se pueden revisar los archivos generados en cada etapa.
- `data/gold/green_data_warehouse.db` es la base que consultan los endpoints del dashboard.
- El endpoint de auditoría de calidad revisa, entre otras cosas, que haya registros, que ciertas claves no se repitan y que las relaciones entre tablas sean válidas.

9. Guion corto para presentar el proyecto

> “Este proyecto integra dos fuentes ambientales de Colombia: el inventario histórico de gases de efecto invernadero y mediciones de calidad del aire de WAQI. Primero conserva los datos originales en Bronze. Después, en Silver, los limpia y los organiza. Finalmente, en Gold, crea tablas y resúmenes para analizarlos en SQLite, mostrarlos en una aplicación web y generar un reporte Excel. La aplicación permite consultar tendencias y, de forma opcional, conversar con un modelo local de Ollama usando un resumen de esos datos.”

Preguntas que podrían hacerte

¿Por qué no guardar todo directamente en SQL?
Se conserva primero una copia de origen para poder rastrear qué llegó. Luego se limpia y modela para que las consultas sean más sencillas y consistentes.

¿La calidad del aire se actualiza continuamente?
No. WAQI se consulta al ejecutar Bronze. Para obtener una medición más reciente hay que volver a ejecutar la extracción y las etapas posteriores.

¿Qué diferencia hay entre Silver y Gold?
Silver corrige y organiza los registros. Gold prepara tablas y resúmenes específicos para consultas, indicadores y reportes.

¿Para qué sirve Ollama?
Para permitir preguntas en lenguaje natural sobre un resumen del Data Warehouse. No es necesario para extraer ni transformar los datos.

¿Qué significa que SQLite sea local?
Que la base de datos está en un archivo del proyecto. No hace falta instalar ni administrar un servidor de base de datos aparte.

10. Glosario rápido

Término	Significado sencillo
ETL	Extraer, transformar y cargar datos
GEI	Gases de efecto invernadero
IPCC	Referencia utilizada para clasificar sectores de emisiones
AQI	Índice que resume la calidad del aire
JSON	Formato común para respuestas de servicios web
CSV	Archivo de texto con datos organizados por filas y columnas
SQLite	Base de datos relacional almacenada en un archivo
API	Interfaz que permite que programas intercambien datos
FastAPI	Herramienta Python usada para crear el backend web
React / Vite	Tecnologías usadas para la interfaz web
Ollama	Herramienta para ejecutar modelos de lenguaje localmente